IntentGate: Action Gating Against Prompt Injection and Tool Poisoning in Tool-Using LLM Agents

Md. Atikur Rahaman [1], Faiaz Hossain Nirob [2], Md. Farid Hossen Rehad [3], Md. Salman Rohoman Nayeem[1], and Md. Aquib Azmain [4]

[1]*Department of Computer Science and Engineering, United International University, Dhaka, Bangladesh*

[2]*Department of Computer Science and Engineering, Independent University Bangladesh, Dhaka, Bangladesh*

[3]*Department of Computer Science and Engineering, Khulna University, Khulna, Bangladesh*

[4]*Department of Computer Science and Engineering, BRAC University, Dhaka, Bangladesh*

***Abstract*—Tool-using agents built on large language models (LLMs) can be redirected by instructions hidden in tool output or tool descriptions. The risk grows as agents reach email, calendars, databases, and code. Existing defenses modify the model, add a judge call per action, or require hand-authored per-tool policies, so enforcement stays costly or incomplete. IntentGate is a lightweight middleware gate that derives an intent contract once from the user's trusted request, before any attacker content is loaded. Every proposed tool call is scored by embedding similarity to the contract plus a deterministic rule veto. The gate then allows, blocks, or escalates the call, with no LLM call per action. It sits at the tool-call boundary, and its policy comes from the user, not from text an attacker can touch. Evaluation uses InjecAgent for indirect injection and MCPTox for tool poisoning. Under static attacks, attack success on InjecAgent falls from 19.0% to 0.7%, and attack-influenced MCPTox calls fall from 20.7% to 10.7%. The gate adds 16–20 ms p95 CPU latency and needs no hand-authored per-tool contracts, compared with 145 for the baseline. A second backbone (qwen3.5:4b) replicates the reduction (27.1% to 2.1%). Benign utility, prompt baselines, and adaptive probes remain preliminary.**

***Index Terms*—action gate, intent contract, LLM agents, MCP, middleware, prompt injection defense, tool poisoning**

I. Introduction

Tool-using LLM agents act through mail, calendars, databases, and code in a ReAct-style loop [1]. That reach is also the attack surface. Instructions hidden in retrieved documents or tool output can redirect the agent through indirect prompt injection [2], [3]. A malicious tool description can redirect it through tool poisoning, documented on real servers of the Model Context Protocol (MCP) [4], [5]. In both cases the agent performs actions the user never authorized, which is the excessive-agency risk in the OWASP Top 10 for LLM applications [6]. A recent survey catalogues attack techniques across agent workflows and protocol layers [7]. Existing defenses change the model, add a judge call, or rely on hand-authored tool policies, and a few derive a policy from the task [8]. IntentGate is a middleware gate that scores every proposed tool call against an intent contract parsed once from the trusted user request, before any attacker content is loaded. It is evaluated on an injection benchmark (InjecAgent) and a tool-poisoning benchmark (MCPTox).

Prior defenses differ in where they enforce control. Model-level methods change the model or its training data. CaMeL [9] wraps the model in a planner-executor design with capability tracking. StruQ [10] and SecAlign [11] fine-tune the model to separate instructions from data, and instruction-hierarchy training [12] teaches models to prioritize trusted instructions. These approaches need model cooperation, and the reviewed literature reports no head-to-head evaluation on InjecAgent or MCPTox. Data-level methods mark or filter untrusted text. Spotlighting [13] transforms it. Guardrail toolkits [14] enforce programmable rails, but the model still makes the final call.

The empirical question is whether a score gate built from a frozen request contract reduces attacks on both benchmark classes under a shared comparison protocol. On the full evaluation, attack success on InjecAgent falls from 19.0% to 0.7%, and attack-influenced MCPTox calls fall from 20.7% to 10.7%. The gate adds 16–20 ms p95 CPU latency and needs no manual per-tool contracts. A ToolGate reimplementation with 145 contracts stays statistically indistinguishable from the unprotected agent.

II. Background and Related Work

A. Threat model

The threat model is a tool-using LLM agent in a ReAct-style loop [1]. Given a trusted user request, the agent proposes tool calls, executes them, observes the results, and repeats until it produces a final answer. Attacker content arrives through tool descriptions (tool poisoning) or through retrieved or returned content (indirect injection). Tool-selection attacks can also make the agent choose an attacker-registered tool even when the user's task is benign [15]. The attacker's goal is to cause the agent to execute a tool call the user did not authorize: data exfiltration, message forwarding, state mutation, or credential access. The defender's goal is to block or escalate such calls while preserving legitimate task completion. The initial user request is assumed trustworthy. Everything after it (tool definitions returned by a registry, tool outputs, injected document text) is potentially attacker-controlled. Attack content ranges from manually written text to optimized adversarial sequences [16], and earlier risk studies describe related failure modes in emulated agent environments [17].

B. Defenses

Model-level defenses track information flow at the model or runtime level. CaMeL [9] wraps the model in a planner-executor design with capability-based taint tracking, so data from attacker-controlled sources cannot reach tool arguments. StruQ [10] fine-tunes the model to separate instructions from data with structured queries, and SecAlign [11] applies preference optimization to the same end. Instruction-hierarchy training [12] teaches the model to prioritize privileged instructions. All of these require model cooperation and, to our knowledge, publish no head-to-head adversarial evaluation on InjecAgent or MCPTox. Data-level defenses mark or transform untrusted text. Spotlighting [13] converts retrieved content into clearly delimited forms. Guardrail toolkits such as NeMo Guardrails [14] enforce programmable rails around the model, but the model still makes the final call. Detection methods add training or inference cost: BIPIA fine-tunes a detector alongside its benchmark [18], and MELON re-executes with masked tools and compares outputs [19].

Judge-based defenses ask a second language model to vet each proposed call [20]. They are flexible, but they add a language-model call on every action, which adds cost and latency. They also introduce a new attack surface, because the judge can be prompt-injected, and produce implicit policies that are hard to audit. Contract-based defenses come closest to where this work enforces control. ToolGate [21] enforces hand-authored *per-tool contracts* with preconditions and post-conditions for each tool at the tool-call boundary, the same chokepoint used here. In this reimplementation the policy source is the weakness. Contracts are written by hand per tool, coverage is incomplete (65/801 MCPTox tools, 8.1% distinct and 37.2% availability-weighted), and attacker-registered tools have no contract. Progent version 3 [8] derives its initial symbolic policy from the user's task and updates it during execution. A solver-based check (SMT) permits narrowing automatically and requires approval for expansion. Its call check is symbolic, while IntentGate freezes its contract and uses embedding scores plus vetoes. ClawGuard [22] derives task-specific constraints from the user's stated objective and enforces them deterministically, with a user confirmation step. It reports results on InjecAgent and MCPTox, so this paper does not claim to be the first to derive policy from the request. Prompt Flow Integrity [23] prevents privilege escalation through agent isolation and data tracking, and it is evaluated on AgentDojo and AgentBench. TrustAgent [24] uses a hand-authored constitution and is evaluated by emulation, not live tool execution. A recent design-pattern survey [25] argues that the safest agent designs never let untrusted data steer privileged actions, which supports the gate placement used in this paper.

AgentDojo [26], InjecAgent [27], and Agent Security Bench [28] provide the injection benchmarks. MCPTox [4] provides the tool-poisoning benchmark, and the MCP specification itself ships no defense against poisoned registrations [5], [29]. Zhan et al. [30] show that adaptive attackers break defenses that look strong under static attacks. The primary claims are therefore scoped to static attacks, and an offline adaptive-attacker probe is reported as preliminary.

C. Research gap

The closest systems differ from IntentGate in policy source and enforcement. ToolGate needs a hand-authored contract for every tool. Progent derives a symbolic policy and updates it during execution. ClawGuard derives task-scoped constraints from the user objective and asks the user to confirm them. Task Shield adds a language-model judge on every action. IntentGate instead parses the trusted request once into a frozen contract and enforces it with a CPU embedding score plus a deterministic veto. It needs no model change, no retraining, no per-action LLM, and no per-tool authoring. The measured contribution is a direct comparison with a ToolGate reimplementation on an injection benchmark and on the covered subset of a tool-poisoning benchmark, with setup cost measured directly (0 vs. 145 contracts).

III. Method

A. System overview and ordering invariant

IntentGate sits between the agent and the executor. Figure 1 shows the pipeline. The ordering invariant is strict. The intent contract is parsed from the trusted user request *before* tool definitions or any attacker content (injected observations, poisoned tool descriptions) are assembled. Tool outputs are never passed to the parser. Every proposed tool call, user-initiated and attacker-influenced alike, passes through `GateMiddleware.execute()` with no bypass path.

B. Intent parser and contract schema

A trusted intent parser (an LLM at temperature 0, JSON mode, one repair retry) reads the user request once per session and emits an intent contract, the request-derived policy used by the gate

$$C = (A, K, H), \tag{1}$$

In this contract, A is the set of allowed (entity, operation, resource) elements, K is a set of numeric and path constraints, and H is a set of high-risk flags. The implementation realizes A as the contract's goals, expected tool categories, and permitted data access, K as argument constraints, and H as side-effect limits. A call that touches an element outside A, violates a constraint in K, or matches a flag in H fails the rule check and triggers the veto. The schema is versioned and frozen for the evaluation. The contract is embedded once per session and reused. Only the proposed call is embedded at decision time, which keeps the agent prompt cacheable and the gate out of the prompt.

C. Scorer

Each proposed tool call c is scored against the contract. The semantic component maps the cosine similarity between the contract embedding e_C and the call embedding e_c to $[0, 1]$:

$$S_{\text{sem}} = \frac{1 + \cos(e_C, e_c)}{2} = \frac{1}{2}\left(1 + \frac{e_C \cdot e_c}{\|e_C\|\,\|e_c\|}\right), \tag{2}$$

TABLE I
RELATED-WORK COMPARISON: POLICY SOURCE, ENFORCEMENT, PER-ACTION LLM USE, BENCHMARKS, AND SETUP COST.

System	Policy source	Enforcement	LLM/action	InjecAgent/ MCPTox	Setup cost
CaMeL	Model-level	Capability taint tracking	No	No	Model change
StruQ	Model-level	Fine-tuned model	No	No	Model change
Task Shield	Task-scoped (LLM-derived)	LLM-as-judge	Yes	No	Per-task LLM calls
ClawGuard	Task-scoped (from user objective)	Deterministic rules	No	Yes	Once per request
Progent	Task policy (LLM-derived)	Symbolic checks, SMT updates	No (updates)	No	Automatic generation, approval
ToolGate	Manual pre/post contracts	Deterministic symbolic	No	No	145 contracts (InjecAgent)
Ours	**Auto-derived from user request**	**Embedding + rule veto**	No	Yes	**0 contracts**

Algorithm 1 Gate decision for one proposed tool call

Require: contract $C = (A, K, H)$ with embedding e_C; tool call c with embedding e_c; parameters α, τ, δ
Ensure: decision d: allow, block, or escalate
$S_{\text{sem}} \leftarrow \frac{1+\cos(e_C, e_c)}{2}$
$S_{\text{rule}} \leftarrow 1$ if c satisfies A and K, else 0
$S \leftarrow \alpha S_{\text{sem}} + (1-\alpha) S_{\text{rule}}$
if $S_{\text{rule}} = 0$ or $S < \tau - \delta$ **then**
    $d \leftarrow$ block ▷ fail-closed hard veto
else if $S < \tau$ **then**
    $d \leftarrow$ escalate ▷ human-review band
else
    $d \leftarrow$ allow
end if
return d

The rule component is deterministic: $S_{\text{rule}} \in \{0, 1\}$. It is 1 when c passes all constraints in K and 0 when the veto fires. The combined score is

$$S = \alpha\, S_{\text{sem}} + (1 - \alpha)\, S_{\text{rule}}, \tag{3}$$

A veto sets $S = 0$ before thresholding. The default is $\alpha = 0.7$. The threshold $\tau = 0.75$ and half-width $\delta = 0.1$ were fixed on a 50-call pilot before any benchmark run. The study reports the operating point and does not claim that the choice is optimal or that it transfers across splits.

D. Gate middleware and escalate band

The middleware decides per call:

$$\text{decision}(c) = \begin{cases} \text{BLOCK}, & S_{\text{rule}} = 0 \text{ or } S < \tau - \delta, \\ \text{ESCALATE}, & S_{\text{rule}} = 1 \text{ and } \tau - \delta \le S < \tau, \\ \text{ALLOW}, & \text{otherwise}, \end{cases} \tag{4}$$

The block case is fail-closed. The escalate case routes to human review, is counted as a block in benchmark mode, and is shown to the user in demo mode. Algorithm 1 lists the full procedure. Of the 583 escalated calls, 60.7% target attacker tools, 14.8% target user tools, and 24.5% target other tools. The gate adds no LLM call per action. Measured p95 per-call latency is 16–20 ms on CPU.

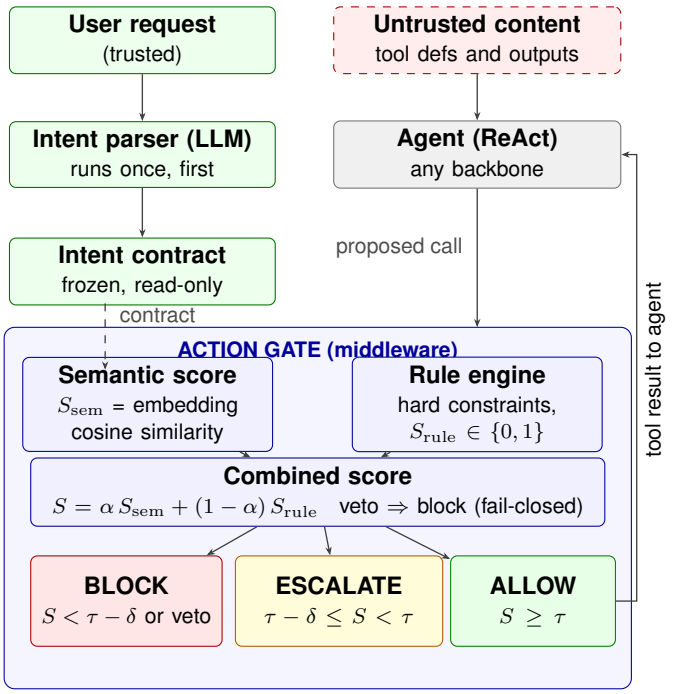


Fig. 1. IntentGate middleware architecture. The intent parser derives a contract once from the trusted request, before any attacker content (tool definitions, tool outputs) is loaded. The gate scores each proposed call with embedding similarity and a hard-rule veto, then decides allow, block, or escalate. Per-action enforcement uses no LLM call.

IV. TOOLGATE BASELINE (B2)

The primary baseline is a minimal reimplementation of ToolGate [21]: contracts with preconditions P and postconditions Q over a symbolic world state, plus snapshot and rollback when a postcondition fails. Contracts cover the evaluated tool subset only: InjecAgent 79/79 user and attacker tools (100%), MCPTox 65/801 registered tools (8.1% distinct and 37.2% availability-weighted). Missing contracts are never silently allowed: they are counted as `no_contract` and reported. Fidelity was validated by contract review and recorded-call replay instead of a ToolBench run.

Setup cost is measured directly: ToolGate requires 145

hand-authored contracts on InjecAgent, compared with zero hand-authored contracts for IntentGate, whose contract is derived from the request. On MCPTox, extending contracts toward 84.9% weighted coverage leaves attack success flat, because attack calls look like legitimate queries. The attack is in the poisoned registration, so the tool call itself looks legitimate. ToolGate is statistically indistinguishable from the unprotected baseline on MCPTox, which supports the direct comparison.

V. Evaluation

A. Benchmarks and cases

InjecAgent [27]: indirect prompt injection in tool-integrated agents, 2,108 cases (direct-harm and data-stealing attacker intents, in base and enhanced settings), frozen at pinned commits. **MCPTox** [4]: tool poisoning on real-world MCP servers, 1,348 static-snapshot cases. MCPTox results are explicitly scoped to the static snapshot (live-server evaluation is deferred, and the poisoning-at-registration logic is identical in both).

B. Metrics

Attack success rate (ASR): share of attack cases where the attacker's objective is achieved. For MCPTox, the same quantity is reported as attack-influenced calls because the heuristic evaluator flags payload or sensitive-path evidence rather than a strict objective. Block rate: share of gated calls blocked or escalated, reported by stratum where stratified. Valid rate: share of user-tool calls the gate allows, a call-level utility proxy. Latency: p50/p95 per-call gate overhead. Setup cost: hand-authored contracts required.

C. Baselines and ablations

B1 is the unprotected agent. B2 is the ToolGate reimplementation. Ours is IntentGate at $\tau = 0.75$, $\delta = 0.1$, $\alpha = 0.7$. The full matrix adds three ablations: A1 semantic-only (no rule veto), A2 rule-only (no embeddings), A3 raw-request embedding (similarity to the request text instead of the parsed contract). All conditions run with identical prompts, tool blocks, agent model (gpt-4o-mini), embedding model (all-MiniLM-L6-v2 [31]), and seed 42.

D. Statistics

All comparisons use the paired McNemar test with continuity correction [32]. Confidence intervals come from a seeded bootstrap with 10k resamples for each benchmark and risk breakdown. The 10 risk breakdowns use Bonferroni correction ($\alpha = 0.005$). Post-hoc power is 1.0 at $\alpha = 0.005$ for both benchmarks. Results from the full matrix are reported as-is. Pilot results are labeled preliminary.

VI. Results

A. Main results

Table II and Figure 2 summarize the headline result. On InjecAgent, IntentGate reduces ASR from 19.0% to 0.7%. On MCPTox, it reduces attack-influenced calls from 20.7% to 10.7%. On the primary slices (the enhanced data-stealing portion of InjecAgent and the full MCPTox snapshot), McNemar gives $p = 1.3\text{e}{-33}$ and $p = 1.25\text{e}{-07}$. The ToolGate baseline reduces ASR by 1.8 percentage points (17.2% vs. 19.0%) on InjecAgent and is not statistically distinguishable from the unprotected agent on MCPTox. Call-level utility is reported with the benign-task pilot in Section VI-F.

TABLE II
Headline results (full evaluation matrix). MCPTox reports attack-influenced ASR. Setup cost counts hand-authored contracts.

Condition	InjecAgent ASR	MCPTox ASR	Setup cost	p95 latency
B1 (none)	19.0%	20.7%	n/a	n/a
B2 (ToolGate)	17.2%	20.1%	145 contracts	0.06 ms
Ours (IntentGate)	**0.7%**	**10.7%**	**0**	16–20 ms

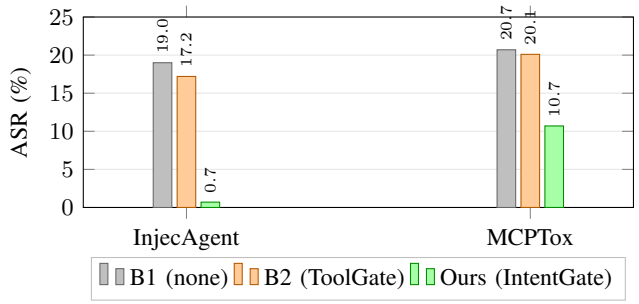


Fig. 2. Attack success rate by condition on the full evaluation matrix (MCPTox reports attack-influenced ASR). IntentGate reaches 0.7% on InjecAgent and 10.7% on MCPTox, while the ToolGate baseline stays close to the unprotected agent.

B. Setup cost

IntentGate requires zero hand-authored contracts. ToolGate requires 145 contracts on InjecAgent and 65 on MCPTox (full matrix). Extending the MCPTox set to 583 ToolGate contracts raised weighted coverage to 84.9% without closing the gap. ToolGate's paper does not report this setup-cost metric.

C. Ablations

Semantic scoring alone reaches 0.0% ASR on InjecAgent. The rule-only ablation (A2) is the weakest condition, with 14.7% ASR on InjecAgent and 263 attack-influenced MCPTox calls, close to the unprotected agent on the hard slices. Semantic-only (A1) detects more than the full gate on MCPTox (81 vs. 144 attack-influenced calls) but over-blocks legitimate calls, as the benign-task pilot shows. The raw-request ablation (A3) matches the full gate on InjecAgent (0.7% both) and catches slightly more attacks on MCPTox (116 vs. 144 attack-influenced calls). The contract's value is therefore utility and auditability. Headline ASR is similar. The false-escalate rate is 14.8% over those 583 escalated calls.

D. Pareto and threshold

Figure 3 shows the threshold sweep on the pilot data: attack success falls to zero at the operating point $\tau = 0.75$ while the false-positive rate stays at 4%. The operating point is fixed

TABLE III
ABLATIONS (FULL EVALUATION MATRIX). A1–A3 REPLACE GATE COMPONENTS. MCPTOX REPORTS ATTACK-INFLUENCED CALLS (OUT OF 1,348).

Condition	InjecAgent ASR	MCPTox calls	Notes
A1 (semantic-only)	0.0%	81	Detects more. Over-blocks legitimate calls
A2 (rule-only)	14.7%	263	Closest to unprotected on hard slices
A3 (raw-request)	0.7%	116	Equal to ours on InjecAgent. Fewer MCPTox calls
Ours (full)	**0.7%**	**144**	Contract adds utility and auditability

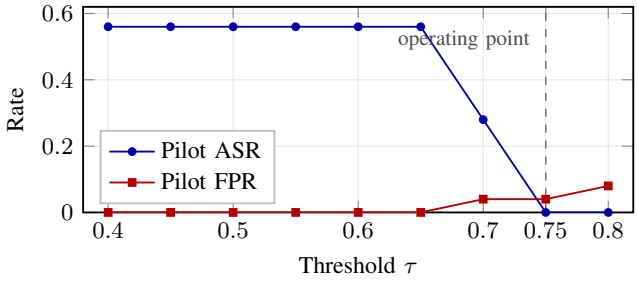


Fig. 3. Threshold sweep on the 200-case pilot ($\delta = 0.1$): attack success rate (ASR) and false-positive rate (FPR) versus τ. The operating point $\tau = 0.75$ sits at the knee: ASR reaches zero while FPR stays at 4%.

for the full matrix, and threshold transfer across encoders and splits is not claimed.

E. Error taxonomy

Thirteen InjecAgent failures remain. All are read-only exfiltration calls with $S \in [0.75, 0.79]$, just above τ. All 13 were already allowed, and rewording them to sound like the user's request raised S by 0.0112 on average without moving any additional case across τ. These prose variants therefore do not change the operating point. The remaining MCPTox failures cluster in poisoned-registration cases whose calls look like legitimate queries, so no request-level rule fires.

F. Validation results (preliminary)

A second backbone, qwen3.5:4b, replicates the ASR reduction on InjecAgent (27.1% $\rightarrow$ 2.1%). gpt-6-luna is excluded because its prose-only output never participates in the ReAct-style loop. On 2,013 gated calls, the block rate is 12.8% for specific contracts, 9.6% for vague contracts, and 12.4% overall.

The benign-task pilot (50 clean cases) compares the unprotected agent, the full gate, and the semantic-only ablation. User-tool allowed rates are 16.0%, 12.0%, and 8.0%. Clean-call block rates are 0.0%, 25.0%, and 42.9%. Semantic-only over-blocks legitimate calls, so the contract preserves utility in this comparison.

The prompt-baseline pilot tests 50 attack cases per condition. ASR is 6.8% with no defense prompt, 4.3% with delimiters, 4.3% with sandwiching, and 8.9% with instructional prevention. DeepSeek V4.1 Flash refused the static injections (0/100 in both conditions), so its delta is undefined. This is a model-level refusal, not a gate effect. Phrasings that change how the injection is delivered stayed at or below 2.2% gated ASR. All pilot numbers are preliminary.

TABLE IV
UTILITY AND VALIDATION PILOTS (PRELIMINARY): BENIGN-TASK UTILITY, SECOND BACKBONE, AND PROMPT-LEVEL BASELINES.

Pilot	Result
Benign utility	user-tool allowed: none 16.0%, ours 12.0%, semantic-only 8.0%. Clean-call block: none 0.0%, ours 25.0%, semantic-only 42.9%
Second backbone	qwen3.5:4b: ASR 27.1% $\rightarrow$ 2.1%
Prompt baselines	delimiters 4.3%, sandwich 4.3%, no defense 6.8%, instructional prevention 8.9% (50 cases each)

VII. LIMITATIONS

The primary claims are scoped to static attacks. Adaptive-attacker performance is not claimed. An offline probe that copies the user request into free-text tool arguments can inflate S above τ. In an offline replay, a six-token verbatim-echo guard blocks all 34 measured adaptive calls (11/11 call templates) with no new blocks among 194 allowed user-tool calls. The main results remain unchanged. MCPTox results are on a static snapshot, not live servers. The full matrix uses a single backbone (gpt-4o-mini). qwen3.5:4b replicates the delta, and gpt-6-luna is incompatible with the shared ReAct-style prompt. DeepSeek's refusal is a model boundary, not evidence of gate generality. The parser is an LLM call once per session, included in the cost. Per-action enforcement uses no LLM.

For the ToolGate baseline, contract coverage for MCPTox tools outside the evaluated subset remains low by design. Missing contracts are counted as `no_contract` and never silently allowed. The valid rate measures call-level allowance of the user's tool call, not end-to-end task success. The benign-task pilot is a 50-case comparison and should be read as such. Block rate includes escalate candidates and is not a pure false-positive rate. The MCPTox attack-influenced metric comes from a heuristic evaluator (payload and sensitive-path evidence), which is coarser than a strict objective check.

VIII. CONCLUSION

The results show that an action gate can derive its policy automatically from the user's request and enforce it with embedding similarity plus a deterministic veto at the tool-call chokepoint. The gate needs no hand-authored per-tool contracts, adds 16–20 ms p95 latency, and reduces attack success on an injection and a tool-poisoning benchmark. The reproducible artifact is released after acceptance. Future work covers adaptive-attacker hardening, live-server MCPTox evaluation, and multi-turn drift.

ACKNOWLEDGMENT

The authors thank their supervisor for supervision, guidance, and continuous support throughout this work. Large language models were used only for language polishing, such as grammar and spelling correction.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[3] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," 2022, NeurIPS 2022 ML Safety Workshop. [Online]. Available: https://arxiv.org/abs/2211.09527

[4] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X.-Y. Li, "MCPTox: A benchmark for tool poisoning on real-world MCP servers," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026, pp. 35 811–35 819. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/40895

[5] B. Radosevich and J. Halloran, "MCP safety audit: LLMs with the Model Context Protocol allow major security exploits," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2504.03767

[6] OWASP Foundation, "OWASP top 10 for large language model applications," 2025, available: https://genai.owasp.org/llm-top-10/.

[7] M. A. Ferrag, N. Tihanyi, D. Hamouda, L. Maglaras, A. Lakas, and M. Debbah, "From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows," *ICT Express*, vol. 12, pp. 353–383, 2026.

[8] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," 2025, preprint, version 3, revised May 14, 2026. [Online]. Available: https://arxiv.org/abs/2504.11703v3

[9] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design (CaMeL)," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2503.18813

[10] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security)*, 2025, pp. 2383–2400. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25

[11] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025, pp. 2833–2847.

[12] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," 2024, preprint. [Online]. Available: https://arxiv.org/abs/2404.13208

[13] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," 2024, preprint. [Online]. Available: https://arxiv.org/abs/2403.14720

[14] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2023, pp. 431–445.

[15] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in LLM agents," 2025, preprint, version 3. [Online]. Available: https://arxiv.org/abs/2504.19793v3

[16] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023, preprint. [Online]. Available: https://arxiv.org/abs/2307.15043

[17] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[18] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2025, pp. 1809–1820.

[19] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable defense against indirect prompt injection attacks in AI agents," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. [Online]. Available: https://arxiv.org/abs/2502.05174

[20] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2025, pp. 29 680–29 697.

[21] Y. Liu, X. Peng, J. Cao, X. Wang, S. Deng, J. Chen, J. Yin, and X. Zhang, "ToolGate: Contract-grounded and verified tool execution for LLMs," 2026, preprint. [Online]. Available: https://arxiv.org/abs/2601.04688

[22] W. Zhao, Z. Li, P. Zhang, and J. Sun, "ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection," 2026, preprint. [Online]. Available: https://arxiv.org/abs/2604.11790

[23] J. Kim, W. Choi, and B. Lee, "Prompt flow integrity to prevent privilege escalation in LLM agents," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2503.15547

[24] W. Hua, X. Yang, M. Jin, Z. Li, W. Cheng, R. Tang, and Y. Zhang, "TrustAgent: Towards safe and trustworthy LLM-based agents," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.585/

[25] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn, "Design patterns for securing LLM agents against prompt injections," 2025, preprint. [Online]. Available: https://arxiv.org/abs/2506.08837

[26] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. [Online]. Available: https://arxiv.org/abs/2406.13352

[27] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-acl.624/

[28] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, Q. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2410.02644

[29] Anthropic, "Introducing the Model Context Protocol," 2024, available: https://www.anthropic.com/news/model-context-protocol.

[30] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 7101–7117. [Online]. Available: https://aclanthology.org/2025.findings-naacl.395/

[31] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3980–3990.

[32] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.